

An Evidence-Driven Fault Classification and Auto-Recovery Framework for OpenAirInterface 5G Standalone Testbeds

*Note: Sub-titles are not captured for <https://ieeexplore.ieee.org> and should not be used

Eueung Mulyana

*School of Electrical Engineering and Informatics
Institut Teknologi Bandung
Bandung, Indonesia
eueung@itb.ac.id*

Ridha Muldina Negara

*School of Electrical Engineering
Telkom University
Bandung, Indonesia
ridhanegara@telkomuniversity.ac*

Ardika Putra Hadian

*School of Electrical Engineering
Telkom University
Bandung, Indonesia
ardikaputrah@telkomuniversity.ac.id*

Neil Justin Surupati

*School of Electrical Engineering
Telkom University
Bandung, Indonesia
neiljustin@telkomuniversity.ac.id*

Sandrina Eka Putri

*School of Electrical Engineering
Telkom University
Bandung, Indonesia
sandrinaekaputri@telkomuniversity.ac.id*

Arga Ulya Abdurrohman

*School of Electrical Engineering
Telkom University
Bandung, Indonesia
argaulayaa@telkomuniversity.ac.id*

Abstract—This document is a model and instructions for aku jawa. This and the IEEEtran.cls file define the components of your paper [title, text, heads, etc.]. *CRITICAL: Do Not Use Symbols, Special Characters, Footnotes, or Math in Paper Title or Abstract.

Index Terms—component, formatting, style, styling, insert.

I. INTRODUCTION

A. Background

With the advent of fifth-generation (5G) mobile networks [1][2], open and flexible research platforms have become increasingly important for both academia and industry. OpenAirInterface (OAI) stands out as one of the most widely adopted open-source solutions for building and evaluating 3GPP-compliant networks, including standalone 5G (SA) deployments, Non-Public Networks (NPNs), and Open Radio Access Network (O-RAN) designs [3][4]. The complete OAI software stack enables fast prototyping independent of proprietary systems, supporting both core network and Radio Access Network (RAN) functions on commodity hardware and software-defined radio [3]. Consequently, research on network optimization, resource allocation, automation, AI-driven control, and private 5G deployment increasingly relies on OAI-based testbeds [5][6][7]. Despite their growing adoption, OAI-based 5G testbeds remain vulnerable to operational failures that jeopardize experiment reproducibility and reliability [4][8]. Common causes of failure include UE registration problems and lingering system states left behind by previous disruptions [6][9]. A major obstacle is that such

failures frequently exhibit similar symptoms handling in OAI-based laboratory networks remains largely manual and highly dependent on operator expertise [10][11]. In practice, operators often prefer performing a full system restart over conducting a systematic root cause analysis. Although this action can restore the system to a functional state, the approach potentially increases recovery time, hinders the identification of the primary cause of failure, and reduces experimental reproducibility [11].

B. Related Works

1) Open-Source 5G Testbeds and OAI-Based Deployments Open-source platforms play a critical role in 5G research due to their flexibility and accessibility [3][5][2]. A notable example is OpenAirInterface (OAI), which has seen extensive adoption in both 4G and 5G experimentation [3][16]. Researchers have deployed 5G networks using OAI on commercial off-the-shelf hardware (COTS) paired with software-defined radio (SDR) platforms [6][4][9]. This track record reflects OAI's practicality and reliability for protocol validity, performance evaluation, and experimental work spanning academic and industrial settings alike.

Throughput, latency, and signal coverage of OAI-based deployments in indoor and laboratory settings have been the subject of several prior investigations [4][8][9]. Separately, comparisons among open-source 5G testbeds under varying software configurations and deployment scenarios have also been reported [8][6]. However, most of these studies focus exclusively on testbed performance under normal, stable operating conditions; the challenges arising when testbeds experience failures remain largely unexamined [10][11]. How OAI-based deployments actually behave once failures set in,

Identify applicable funding agency here. If none, delete this.

however, remains poorly documented.

2) O-RAN Automation and Intelligent Network Operation The emergence of O-RAN has spurred a substantial body of research into programmable, self-managing networks by enabling new forms of automation and intelligent control. Central to this capability is the Near-RT RIC. Combined with xApps and policy-driven control loops, it gives the network a means of responding dynamically to changing conditions, a capability that conventional RAN architectures do not offer [17][2][13].

Several testbeds built on this foundation have explored AI-driven monitoring and anomaly mitigation within O-RAN environments [7], showing that radio-access anomalies can be detected, runtime telemetry collected, and intelligent policies applied to shape network behavior [18][12]. The bulk of this work, however, remains concentrated on radio-level concerns and RIC-based optimization. Failures occurring deeper in the protocol, stack, such as routing inconsistencies, user-plane disruptions, tunnel failures, or lingering system states, receive comparatively little attention, even though they are frequently encountered in experimental 5G deployments [10][11].

3) Reliability and Recovery in Experimental 5G Platforms Any experimental networking platform depends fundamentally on reliability. An unstable testbed does more than interrupt ongoing experiments; it also calls into question the validity and repeatability of the results obtained. While existing deployment frameworks emphasize reproducibility through standardized scenarios and validation procedures, most of them assume that the underlying infrastructure is already operational [9][6].

In practice, however, failures do occur. When they arise in laboratory environments, recovery typically falls to experienced operators who rely on trial-and-error or full-system restarts rather than structured fault diagnosis [10][11]. Although this approach may restore the system to an operational state, it provides little insight into the actual root cause, and repeated ad hoc recovery makes experimental results harder to reproduce. Moreover, unnecessary restarts can introduce additional downstream complexity and obscure the connection between a fault and its remedy.

4) Research Gap and Positioning of This Work Based on the reviewed literature, existing studies generally fall into three groups: (i) deployment and implementation of open-source 5G platforms, (ii) deployment and implementation of open-source 5G platforms, and (iii) O-RAN automation and AI-assisted network control. Although these works have substantially advanced open-source cellular research, operational reliability management within OAI-based 5G Standalone testbeds remains largely overlooked [10][11].

Specifically, no existing work integrates fault characterization, evidence-driven diagnosis, structured recovery policies, and repeatable operational monitoring into a unified framework. Most prior studies evaluate network performance only after a successful deployment, leaving the processes needed to

detect, classify, and recover from failures largely unaddressed [4][8].

To address this gap, this paper focuses on operational reliability in an OpenAirInterface-based 5G SA/SNPN testbed. Unlike prior works centered on performance benchmarking or deployment automation, this work introduces an evidence-driven fault characterization framework, a minimal-first recovery policy, and a watchdog-assisted operational workflow, all designed to improve recovery effectiveness and experiment repeatability. Together, these contributions aim to establish a reliability layer that supports future research in AI-assisted network operation and O-RAN automation.

C. Contributions and Limitations

To address the gaps identified above, this paper proposes an evidence-driven operational reliability framework for OAI-based 5G Standalone Non-Public Network (SNPN) testbeds [1][3]. This framework incorporates structured health monitoring, fault characterization, evidence-based fault classification, and recovery rules that prioritize minimal intervention into a scalable operational workflow [10][11]. Instead of triggering an immediate restart of every component, the framework instead seeks out the smallest set of remedial actions sufficient to restore system functionality. Faults are classified from observable runtime indicators, including registration status, routing state, user-plane connectivity, and protocol-level signals, to guide targeted recovery [6][9]. Additionally, a watchdog-assisted monitoring mechanism continuously records machine-readable artifacts, supporting both automated decision-making and post failure analysis [11][14].

The following summarizes the main contributions of this paper as follows.

1) A fault characterization framework covering common operational failures in OAI-based 5G SA/SNPN testbeds, including routing, user-plane, registration, and infrastructure-related fault conditions. 2) An evidence-driven, minimal-first recovery mechanism that classifies faults from observable indicators to select targeted corrective actions over blind full-system restarts. 3) A structured operational workflow integrating watchdog monitoring and machine-readable artifacts to improve experiment repeatability and operational transparency. 4) An experimental evaluation conducted on a two-node OAI 5G SA testbed, in which fault detection capability, recovery effectiveness, and operational repeatability were assessed across a set of representative fault scenarios.

Experimental results demonstrate that the proposed approach reliably detects representative operational faults and restores system functionality with fewer unnecessary recovery actions. The evidence-based recovery policy provides a practical foundation for improving the reliability of open-source 5G testbeds and servers as an enabling layer for future research in AI-assisted network operations and O-RAN automation [14][15].

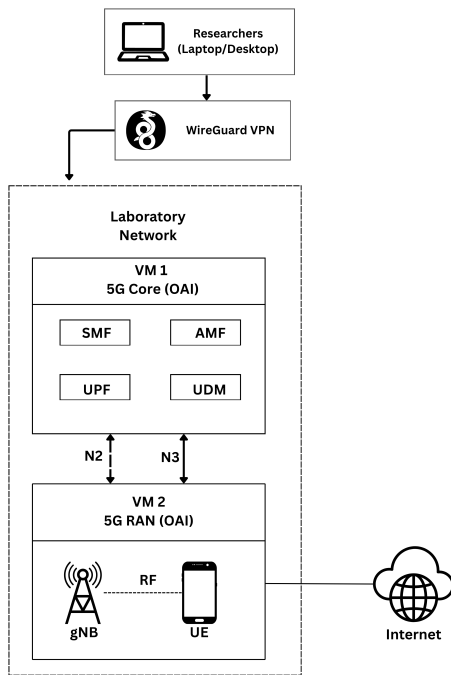


Fig. 1. OAI Based 5G SA/SNPN Testbed Architecture

D. Paper Organization

II. METHODS

The system architecture, success-gate definition, evidence sources, fault taxonomy, recovery workflow, experimental setup, compartmented methods, fault injection scenarios, procedure, sample-size rationale, and statistical analysis used in this study are presented in the following subsections.

A. System Architecture

1) OAI 5G SA/SNPN Testbed Architecture

An OAI-nased 5G Standalone (SA) Shared and Non-Public Network (SNPN) testbed, deployed on two laboratory virtual machines, served as the experimental platform for the work reported here. The platform was designed to offer a controlled, repeatable environment in which operational reliability, fault behaviour, and automated recovery in open-source 5G deployments could be examined under realistic conditions.

The testbed is composed of two nodes. The core node runs the OAI 5G Core Network Functions responsible for mobility management, session establishment, and user-plane services. The RAN node runs OAI gNB and UE instances, both operating in Radio Frequency Simulator (RFSim) mode so that signaling and protocol behavior can be preserved without any dependency on physical radio hardware. Interaction between the Core and the RAN follows the standard 5G Core Network functions responsible for mobility management, session establishment, and user-plane for registration and data connectivity, which keeps the environment repeatable across runs while removing the variability that physical RF

conditions would otherwise introduce.

The testbed is reached from the local laboratory network, or remotely through a WireGuard VPN tunnel after which SSH is used to operate both virtual machines. With this configuration, monitoring, health checks, fault injection, and recovery can be carried out without anyone needing to be physically present in the laboratory, which made the volume of repeated experimentation in this study practical.

2) Success Gate and Health Indicators Assessing whether an OAI-based 5G testbed is operational cannot be reduced to checking individual components. since a gNB can register successfully with the core while the UE still fails to obtain user-plane connectivity. A success gate was therefore introduced as a single combined criterion for testbed health, rather than relying on any one component status.

Under this definition, the testbed counts as healthy only when every gate condition holds at the same time; if even one condition fails, the state is treated as unhealthy and may trigger recovery.

Table 1. Success Gate Indicators. The four indicators cover both control-plane and user-plane behavior. AMF association confirms that the gNB has registered with the core network. UE registration is verified by the appearance of FGS_REGISTRATION_ACCEPT in the UE log [19]. Tunnel availability checks that oaitun_ue1 exists the correct IP configuration, and Internet reachability checks that traffic actually passes through that panel.

A pass on all four indicators is labeled SUCCESS_GATE = PASS; any failure is labeled SUCCESS_GATE = FAIL. This binary label is what the rest of the framework – fault detection, recovery verification, watchdog monitoring – is built around.

3) Evidence Sources and Fault Taxonomy

A single log or indicator rarely tells the whole story, since different underlying faults can produce nearly identical symptoms. The diagnostic logic in this study therefore draws on several sources at once, spanning control-plane, user-plane, routing, connectivity, and process-level observations. Table 2 A loss of internet connectivity, for instance, could stem from a routing problem, a broken tunnel, a user-plane failure, or a dead service process. Cross-checking across sources before committing to a fault label cuts down on this kind of ambiguity and avoids triggering recovery actions on incomplete information. Faults observed during experimentation were grouped into a fixed taxonomy rather than treated as one undifferentiated category of "service down."

Table 3. Fault Taxonomy

Routing faults disrupt packet forwarding without necessarily touching registration. User-plane faults break data connectivity even when control-plane signalling still works. Registration faults block attachment outright, and infrastructure faults trace back to a failed service or process rather than the network logic itself. Each fault category is tied to a recognizable

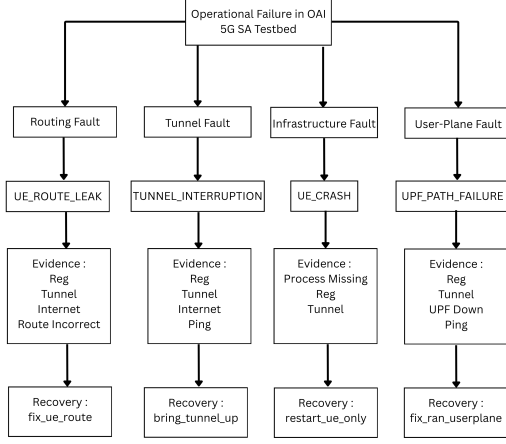


Fig. 2. Operational Failure Handling in OAI 5G Testbed

evidence pattern, derived from repeated observation during testing.

Table 4. Example Evidence-to-Fault Mapping.

These mappings, let the recovery engine act on a named fault category instead of a raw log output, so that each recovery decision can be traced back to specific observable evidence rather than guesswork.

B. Workflow

1) Operational Reliability Framework

Manual troubleshooting on a 5G testbed is slow and inconsistent, which motivated the design of an evidence-driven reliability framework that watches testbed health continuously, gather evidence when something looks wrong, classifies the likely fault, and applies a recovery action chosen from policy, targeted fixes over a blanket restart.

Five stages make up the loop. Health monitoring runs continuously using predefined success-gate and flags degradation early. Once an anomaly appears, evidence collection pulls together registration logs, routing tables, tunnel status, connectivity tests, and process state. That evidence feeds fault classification, which maps the observed pattern onto one of the predefined fault categories instead of lumping everything under a generic failure. The recovery policy engine then picks a corrective action matched to that classification, starting from the lightest intervention and escalating only if needed. Finally, verification re-checks the success gate; if the testbed comes back healthy the loop closes, and if not, the fault is still considered active and may be escalated further.

Algorithm 1 lays out this classification and recovery logic in pseudocode.

Algorithm 1. Evidence-Driven Fault Classification and Minimal-first Recovery.

2) Experimental Evaluation Setup

To test whether the framework above actually helps, its recovery behavior was compared against a blind full restart, a "kill everything and start over" approach that operational teams commonly fall back on when no fault classification is available, and one that also shows up frequently in related work. Comparing against a human operator was considered but ultimately set aside, since human response time is hard to standardize and replicate under laboratory conditions; the blind-restart baseline gives a fixed, repeatable reference point for both recovery time and action count instead. Three things were of particular interest going into the experiments: whether the classifier could reliably pin down injected faults using the available indicators (registration log, routing table, tunnel status, ping results); whether the minimal-first approach actually beat blind restart on recovery time and action count, across faults of differing severity; and whether recovery outcomes stayed consistent when the same fault scenario was repeated many times.

3) Testbed Configuration

Experiments ran on the two-node testbed introduced in Section II-A1. Both nodes are virtual machines on a shared Proxmox host, with CPU and memory left uncommitted to other workloads during testing. The internal network sits on a Proxmox bridge using subnet 172.20.0.0/16; for the June 2026 experiment session, the core node was assigned 172.20.0.110 and the RAN node 172.20.0.101, with the Proxmox gateway at 172.20.0.1. Because IP assignments can shift after a reboot or topology change, addresses were re-checked before each session.

The OAI 5G Core runs on the core node as Docker containers vi a docker-compose-basic-nrf.yaml. The gNB and UE run directly on the RAN node without containerization, using the nr-softmodem and nr-uesoftmodem binaries in -rfsim mode. The gNB configuration file (configpasti.conf) sets amf_ip_address to 172.20.0.110, GNB_IPV4_ADDRESS_FOR_NG_AMF to 172.20.0.101, sdr_addrs to "type=rfsim=", and PLMN values mcc=208, mnc=95, sst=1. The UE configuration (ue_pasti.conf) sets -ue-nb-ant-rx- 2 -ue-nb-ant-tx 2 to meet RFSIM's radio front-end requirements.

A handful of network settings need to persist for the system to work end to end: a static route to 192.168.70.128/26 via 172.20.0.110, an iptables DNAT rule for 192.168.70.134:2152, a UDP 2152 checksum fill, PREROUTING NOTRACK, and FORWARD ACCEPT. Because VM reboots wipe out ephemeral iptables rules and routes, these are reapplied automatically every 15 seconds through a system times on each node, namely oai-ran-netfix.timer on the RAN side and oai-core-netfix on the core side.

In line in the definitoin in Section II-A2, a run is treated

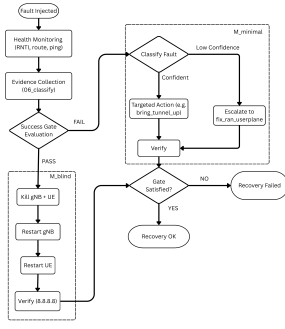


Fig. 3. Recovery Decision Workflow for M_blind and M_minimal Methods

as healthy only when all four condition hold together: the gNB is associated with the AMF over N2; the UE has received FGS_REGISTRATION_ACCEPT; the tunnel interface oaitun_ue1 exists with a valid address in 12.1.1.0/24; and four-packet ICMP ping over oaitun_ue1 to an external host (8.8.8.8) comes back with 0% loss. This four-part gate served as the ground truth for every run [19].

4) Compared Methods

Two recovery methods were tested under matching fault conditions. M_blind, the blind full restart, runs a fixed sequence no matter what fault was injected: kill all gNB and UE processes (killall -9 nr-softmodem nr-uesoftmodem), clear the tunnel interface if it still exists, restart the gNB and wait 25 seconds for it to associate with the AMF, restart the UE and wait 30 seconds for FGS_REGISTRATION_ACCEPT plus tunnel re-establishment, then confirm connectivity with a ping. It reads no diagnostic evidence at any point. Instead, the same sequence, timing, and number of recovery actions are applied regardless of the fault type, making it a deterministic baseline. The script is bin/control/blind_restart.sh.

M_minimal, the evidence-driven method, follows the proposed evidence-driven logic. Before doing anything corrective, it gathers tunnel interface status (ip addr show oaitun_ue1), the routing entry toward the external host (ip route get 8.8.8.8), ping results on both the default route and tunnel path, gNB/UE process status, and the AMF association count from the gNB log. The resulting pattern is matched against the defined fault categories, and the smallest viable corrective action is pulled from the evidence-to-fault mapping. For example, a routing device of eth0 with the tunnel still intact maps to 'fix_ue_route' (one action about 7 s); a tunnel interface in DOWN state maps to 'bring_tunnel_up' (one action, about 2 s); an absent UE process maps to 'restart_ue_only' (one action about 25 s). When the evidence does not clearly point to one category, the method escalates to 'fix_ran_userplane'. This script is 'bin/control/autoheal_minimal.sh'.

Both methods are wrapped as shell scripts that take 'fault_type' and 'run_id' as arguments, which lets the experiment launcher run through all 275 runs without manual intervention. Each run writes a 'summary.json' containing 'run_id', 'method', 'fault_type', 'recovery_time_sec',

'actions_count', 'actions_taken', 'success', and 'ue_ip'. This fixed schema simplifies cross-method and cross-fault aggregation during the subsequent analysis.

5) Fault Scenarios and Injection

Four scenarios were chosen from the taxonomy in Section II-A3, one from each of the routing, tunnel, infrastructure, and user-plane categories. Selection followed two rules: the fault had to be injectable and reversible without manual intervention, and its expected fix had to differ meaningfully from a blind restart. Otherwise, the comparison between the two methods would provide limit insights

Table 5. Fault Injection Scenarios.

The baseline restoration timer was paused before each injection and resumed only after recovery had completed. Otherwise, it would have automatically reversed the injected fault during the measurement, thereby skewing the recorded recovery times.

Each injection was confirmed effective before any recovery step began: UE_ROUTE_LEAK by checking that 'ip route get 8.8.8.8' reports eth0 as the device; TUNNEL_INTERRUPT by confirming the interface state is DOWN; UE_CRASH by confirming 'pgrep -x nr-uesoftmodem' returns nothing; UPF_PATH_FAILURE by confirming 'ip route — grep 192.168' returns nothing. Runs where the fault could not be confirmed were flagged, pulled from the main analysis, and reported on their own.

6) Experimental Procedure and Sample Size

Every run followed the same ten steps: confirm a healthy baseline through the success gate; pause the restoration timer on both nodes; inject the target fault per Table 5; wait 3 seconds for it to settle; confirm the fault took effect; run the assigned recovery method and time it from the first recovery action to a successful ping; re-check the success gate to confirm full recovery; resume the restoration timer; wait another 3 seconds before moving on; and write the 'summary.json' plus pre-step artifact logs. A single M_minimal run took 5-15 seconds depending on which action fired, while M_blind consistently took 75-80 seconds, most of it spent waiting through the mandatory 25-second gNB restart and 30-second UE registration windows. The full dataset combines two batches: Batch 160 (n=160, runs.csv) and a rerun, Batch 161 (n=115, runs_rerun.csv), for 275 in total. The original plan called for 20 runs per cell (4 faults x 2 methods) [20], or 160 runs overall; the rerun added another 115 to firm up per-cell stability, leaving 27-41 successful runs in each cell, which is comfortably above the n=21 per-group threshold needed for Mann-Whitney testing at $\alpha = 0.05$ with a moderate effect size (Cohen's ≈ 0.8) [21][22]. Because the two methods ended different sample sizes (M_blind n=130, M_minimal n=145), a balanced subsample of 260 runs (130 per method, seed=42) was also drawn by randomly dropping 15 M_minimal runs, including three each from TUNNEL_INTERRUPT, UE_ROUTE_LEAK, and UPF_PATH_FAILURE, and six from UE_CRASH. The main results below use the full 275-run dataset throughput; the balanced subsample is reported separately, purely as a

robustness check. Running all 275 trials took roughly 5-6 hours of walk-clock time in total.

Runs were executed one after another rather than in parallel, since both methods touch shared testbed state: routing entries, iptables rules, and running processes. Running them concurrently would have required isolation infrastructure that the target deployment does not actually have. Statistical comparisons uses the Mann-Whitney Utest with Bonferroni correction (adjust $\alpha = 0.0125$ across the four faults) and Cliff's delta as the effect-size measure; success-rate comparisons used Fisher's exact test [21].

7) Performance Metrics and Statistical Analysis

Three metrics anchor the evaluation: recovery time, success rate, and action count. Recovery time is the gap between starting recovery and reaching a healthy state again:

$$t_{\text{recovery}} = t_{\text{healthy}} - t_{\text{recovery,start}} \quad (1)$$

where $t_{\text{recovery,start}}$ marks the first recovery action and t_{healthy} marks the point at which every success-gate condition is met. Success rate is simply:

$$SR = \frac{N_{\text{success}}}{N_{\text{total}}} \times 100\% \quad (2)$$

with N_{success} the number of runs that recovered and N_{total} the total number attempted. Average recovery time and average action count per method are:

$$\bar{T}_m = \frac{1}{n} \sum_{i=1}^n t_i, \quad \bar{A}_m = \frac{1}{n} \sum_{i=1}^n a_i \quad (3)$$

where n is the number of repetitions and a_i the action count for run i . The speedup of M_{minimal} over M_{blind} is expressed as:

$$S = \frac{\bar{T}_{\text{blind}}}{\bar{T}_{\text{minimal}}} \quad (4)$$

Since recovery times are unlikely to be normally distributed, significance was tested with the Mann-Whitney U statistic:

$$U = n_1 n_2 + \frac{n_1(n_1 + 1)}{2} - R_1 \quad (5)$$

where n_1, n_2 are the two sample sizes and R_1 is the rank sum of the first sample. Effect size came from Cliff's Delta, defined in terms of the sign function

$$\text{sgn}(x) = \begin{cases} +1, & x > 0 \\ 0, & x = 0 \\ -1, & x < 0 \end{cases} \quad (6)$$

as

$$\delta = \frac{1}{n_1 n_2} \sum_{i=1}^{n_1} \sum_{j=1}^{n_2} \text{sgn}(x_i - y_j) \quad (7)$$

where $\text{sgn}(\cdot)$ is the sign function defined in (6). Together these numbers capture how effective, efficient, and repeatable the recovery process actually was across fault scenarios.

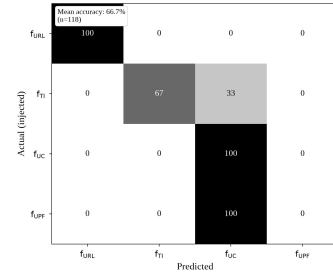


Fig. 4. Fault Classification Confusion Matrix (n=16)

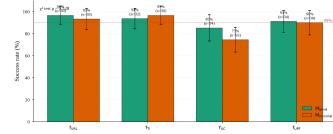


Fig. 5. Recovery Success Rate by Fault Type and Method

The evaluation results, including recovery-time distributions, success rate, action counts, and the corresponding statistical analysis, are presented in Section III.

III. RESULT AND DISCUSSION

Three aspects of the proposed framework are examined here: how reliably the fault classifier identifies injected failures, how the two recovery methods compare against each other in terms of success rate, recovery time, and action count, and whether recovery outcomes stay stable when the same fault scenario is repeated.

A. Fault Classification Accuracy

UE_ROUTE_LEAK and UE_CRASH are both classified with perfect accuracy (100% each). TUNNEL_INTERRUPT is partly mistaken for UE_CRASH (33.3%), and UP_PATH_FAILURE proves the hardest case, with all instances being labeled as UE_CRASH (100%). The macro-averaged per-class accuracy reaches 66.7%, while the overall (micro/weighted) accuracy is 87.5%, both computed across 16 held-out classified runs. The classifier therefore picks out most fault categories correctly, with confusion concentrated mainly between the use-plane and tunnel-related fault types.

B. Recovery Success Rate Comparison

This part of the evaluation checks whether M_{minimal} matches or exceeds the success rate of M_{blind} and M_{minimal} across all four fault types, with the 90% target threshold indicated for reference.

Both methods meet or come close to the 90% target for most fault types. M_{blind} reaches an overall success rate of 92.5% (n=130), peaking at UE_ROUTE_LEAK (96.7%, n=30). M_{minimal} reaches 86.2% overall (n=145), with its best result at TUNNEL_INTERRUPT (96.3%, n=27). The lowest figure belongs to M_{minimal} on UE_CRASH (75.5%, n=49), a drop traced back to the classifier confusing

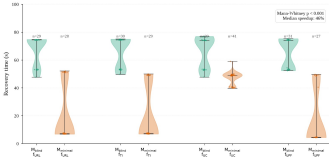


Fig. 6. Recovery Time Distribution by Fault and Method

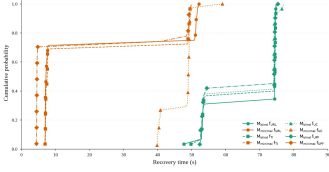


Fig. 7. Empirical CDF of Recovery Time for M_blind and M_minimal Across Fault Types

UE_CRASH with UPF_PATH_FAILURE. A X^2 test gives $p = 0.28$, so the overall difference in success rate between the two methods is not statistically significant.

C. Recovery Time Comparison

Whether M_minimal recovers faster than M_blind is the focus here. Fig. 6 shows how recovery time is distributed for each fault type and method, with M_minimal posting a 46% lower median time overall (M_blind pooled median ~ 74 s versus M_minimal pooled median ~ 20 s; Mann-Whitney $p < 0.001$, Cliff's $\delta = 0.977$).

M_minimal finishes recovery faster across every fault category. For UE_ROUTE_LEAK and TUNNEL_INTERRUPT, it takes roughly 7-22 seconds, against 67-75 seconds for M_blind. UE_CRASH and UPF_PATH_FAILURE shows wider spread under M_minimal because of occasional escalation, yet most runs still finish ahead of M_blind. Pooling across fault types, M_blind sits at a median of ~ 74 s while M_minimal sits at approximately 20 s, a 46% speedup (Mann-Whitney $p < 0.001$, Cliff's $\delta = 0.977$).

Fig. 7 gives the empirical cumulative distribution of recovery time across fault types, which further illustrates how much faster M_minimal recovers compared with M_blind.

The CDF reveals two distinct clusters for M_minimal: roughly 70% of runs finish at 5-8s through single-action recovery, while the remaining approximately 30% sit at 47-52s after escalating to fix_ran_userplan. M_blind, by contrast, forms a single cluster at 47-78s across all fault types. The gap between the M_minimal (teal) and M_minimal (orange) curves holds steady across every fault category, reinforcing the time advantage of the evidence-driven approach.

D. Recovery Action Count Comparison

Whether M_minimal requires fewer recovery actions than M_minimal is examined here. Fig. 8 reports the mean number of actions per recovery and their composition for each fault type; M_minimal needs fewer actions than M_blind in every case (Mann-Whitney $p < 0.001$).

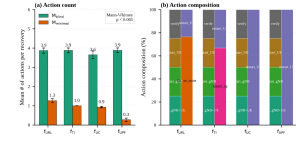


Fig. 8. Recovery Action Count and Composition by Fault and Method

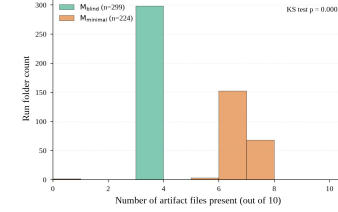


Fig. 9. Distribution of Artifact File per Run Folder for M_blind and M_minimal

M_blind runs through roughly 4 action per recovery no matter the fault, since its restart sequence (kill_gNB, start_gNB, start_UE, verify) is fixed. M_minimal cuts this down sharply, averaging between 0.3 actions (UPF_PATH_FAILURE) and 1.3 actions (UE_ROUTE_LEAK) per run, since only the single action matched to the classified fault is fired. Where escalation kicks in (around 30% of runs), the count rises to 2-3. This pattern reflects the contrast between M_minimal's targeted, minimal interventions and M_blind's blanket restart.

E. Repeatability and Evidence Completeness

Stability of recovery outcomes across repeated runs of the same fault scenario is the focus here. Fig. 9 plots the number of artifact files per run folder, drawn from an audit of 523 run folders covering all 275 CSV runs plus rerun and duplicate folders, and points to a statically significant gap in artifact counts between the methods (KS test $p < 0.001$).

M_blind folders hold 4 artifact file every time, while M_minimal folders hold 6-8, reflecting the extra classification and evidence records the evidence-driven approach generates. This difference is statistically significant (KS test $p < 0.001$), and the audit trail confirms that both methods leave complete, machine-readable records suitable for post-failure analysis and for reproducing the experiment.

F. Sub-Sample Robustness Analysis

Because M_blind ($n=130$) and M_minimal ($n=145$) ended up with unequal sample sizes, a balanced sub-sample of 130 runs per methods (260 runs total, seed=42) was drawn from the full 275 run dataset to check whether this imbalance affected the conclusions.

The balanced results closely track the full dataset: overall success rate comes out at 88.8% versus 88.7% for the full sample ($\Delta = 0.1$ pp), and per-method rates stay unchanged (M_blind 91.5%, M_minimal 86.2%). M_blind's sample size and success rate are identical in both datasets ($n=130$, SR=91.5%), since the balancing procedure only trimmed M_minimal runs. With a negligible effect size for success

rate (Cliff's $\delta = 0.977$), the main conclusions of this study holds up under sample-size imbalance and should generalize to balanced designs.

The same concerns about unequal method-level sample sizes in the primary analysis ($n = 275$) was checked by re-running the comparisons on this balanced subsample ($n = 260$, 130 per method). Table 6 and 7 list the recomputed per-method statistics and the corresponding effect sizes alongside the balanced figures. Direction and magnitude are both preserved ($\Delta\delta \leq 0.04$ across all pairs), which indicates the main findings are not an artifact of sample-sizes imbalance.

Table 6. Balanced subsample analysis ($n=260$, 130 per method). Figures reflect the full sample ($n = 272$) after stratification to give equal representation across methods. The two leftmost columns give mean and standard deviation of the primary metric, while the two rightmost columns compare method-level distribution. The direction and size of the differences hold steady compared with the full sample ($n = 275$), showing the main findings are not driven by sample-size imbalance across methods.

Table 7. Robustness of effect size under balanced subsampling ($n=260$). Cliff's δ is computed on both the full sample ($n = 275$) and the balanced subsample ($n = 260$) for each pairwise comparison. Effects with $|\delta| \geq 0.147$ are conventionally interpreted as non-negligible, and the near-identical δ values across both columns confirm that full-sample conclusions are not artifacts of unequal method representation. The largest shift ($\Delta\delta = 0.004$) stays well inside the small-effect range, which supports the stability of the reported rankings.

Subsampling drew runs without replacement, stratified by method (seed = 42). Cliff's δ values are reported with 95% confidence intervals, with the full numerical output available in subsample_stats.txt (supplementary material). The implication of these results, together with their placement within the broader literature on operational reliability and auto-recovery for OAI-based 5G testbed, are examined in the closing part of this paper.

IV. CONCLUSION

A. Conclusion

An evidence-driven fault classification and minimal-first auto-recovery framework for OAI-based 5G SA/5G-LTE testbeds has been presented in this paper. The investigation finds that structured health monitoring, combined with multi-source evidence collection and a predefined fault taxonomy, can reliably replace ad-hoc, experience-dependent troubleshooting in open-source 5G laboratory environments.

The evidence-driven fault classifier reaches an overall (micro-weighted) accuracy of 87.5% on a held-out subset of $n=16$ classified runs (macro-averaged per-class accuracy: 66.7%), with perfect per-class accuracy for UE_ROUTE_LEAK and UE_CRASH (100% each), and primary ambiguity is the misclassification of both tunnel-related (TUNNEL_INTERRUPT, 33.3%) and user-plane

(UPF_PATH_FAILURE, 100%) faults toward the infrastructure category (UE_CRASH), owing to overlapping downstream escalation paths.

M_minimal consistently outperforms M_blind in recovery time across all four fault types, delivering a median speedup of 46% (Mann-Whitney $p < 0.001$, Cliff's $\delta = 0.977$) and reducing the mean number of corrective actions from approximately 4 to a median of 1 per recovery, with no statistically significant difference in overall success rate between the two methods ($p = 0.28$). Third, both methods produce consistent artifact counts across 523 audited run folders, confirming that the operational workflow yields machine-readable records that support post-failure analysis and experiment reproducibility.

Evidence-driven, minimal-first recovery thus offers a concrete and reproducible alternative to ad-hoc troubleshooting in open-source 5G laboratory environments.

B. Future Work

Several questions opened by this work warrant further investigation. Future work needs to establish whether the fault classifier can be extended with machine-learning techniques to resolve ambiguity between overlapping fault categories – particularly the UE_CRASH and UPF_PATH_FAILURE classes – and to assess the scalability of the framework to multi-node and multi-tenant O-RAN deployments. Additionally, integration with Near-RT RIC xApps and AI-assisted network control loops represents a promising direction for advancing toward full autonomous 5G network operations.

ACKNOWLEDGMENT

The preferred spelling of the word “acknowledgment” in America is without an “e” after the “g”. Avoid the stilted expression “one of us (R. B. G.) thanks ...”. Instead, try “R. B. G. thanks...”. Put sponsor acknowledgments in the unnumbered footnote on the first page.

REFERENCES

Please number citations consecutively within brackets [1]. The sentence punctuation follows the bracket [2]. Refer simply to the reference number, as in [3]—do not use “Ref. [3]” or “reference [3]” except at the beginning of a sentence: “Reference [3] was the first ...”

Number footnotes separately in superscripts. Place the actual footnote at the bottom of the column in which it was cited. Do not put footnotes in the abstract or reference list. Use letters for table footnotes.

Unless there are six authors or more give all authors' names; do not use “et al.”. Papers that have not been published, even if they have been submitted for publication, should be cited as “unpublished” [4]. Papers that have been accepted for publication should be cited as “in press” [5]. Capitalize only the first word in a paper title, except for proper nouns and element symbols.

For papers published in translation journals, please give the English citation first, followed by the original foreign-language citation [6].

REFERENCES

- [1] G. Eason, B. Noble, and I. N. Sneddon, "On certain integrals of Lipschitz-Hankel type involving products of Bessel functions," *Phil. Trans. Roy. Soc. London*, vol. A247, pp. 529–551, April 1955.
- [2] J. Clerk Maxwell, *A Treatise on Electricity and Magnetism*, 3rd ed., vol. 2. Oxford: Clarendon, 1892, pp.68–73.
- [3] I. S. Jacobs and C. P. Bean, "Fine particles, thin films and exchange anisotropy," in *Magnetism*, vol. III, G. T. Rado and H. Suhl, Eds. New York: Academic, 1963, pp. 271–350.
- [4] K. Elissa, "Title of paper if known," unpublished.
- [5] R. Nicole, "Title of paper with only first word capitalized," *J. Name Stand. Abbrev.*, in press.
- [6] Y. Yorozu, M. Hirano, K. Oka, and Y. Tagawa, "Electron spectroscopy studies on magneto-optical media and plastic substrate interface," *IEEE Transl. J. Magn. Japan*, vol. 2, pp. 740–741, August 1987 [Digests 9th Annual Conf. Magnetism Japan, p. 301, 1982].
- [7] M. Young, *The Technical Writer's Handbook*. Mill Valley, CA: University Science, 1989.
- [8] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," 2013, arXiv:1312.6114. [Online]. Available: <https://arxiv.org/abs/1312.6114>
- [9] S. Liu, "Wi-Fi Energy Detection Testbed (12MTC)," 2023, *gitHub repository*. [Online]. Available: <https://github.com/liustone99/Wi-Fi-Energy-Detection-Testbed-12MTC>
- [10] "Treatment episode data set: discharges (TEDS-D): concatenated, 2006 to 2009." U.S. Department of Health and Human Services, Substance Abuse and Mental Health Services Administration, Office of Applied Studies, August, 2013, DOI:10.3886/ICPSR30122.v2
- [11] K. Eves and J. Valasek, "Adaptive control for singularly perturbed systems examples," *Code Ocean*, Aug. 2023. [Online]. Available: <https://codeocean.com/capsule/4989235/tree>

IEEE conference templates contain guidance text for composing and formatting conference papers. Please ensure that all template text is removed from your conference paper prior to submission to the conference. Failure to remove the template text from your paper may result in your paper not being published.